



Arquitetura de Software

Herysson R. Figueiredo
herysson.figueiredo@ufn.edu.br



O que é Arquitetura?

Arquitetura de software representa:

A estrutura do sistema, seus componentes, suas interações e as decisões de projeto que definem seu funcionamento.

- Pressman destaca que é a estrutura de dados e componentes + inter-relações.
- Sommerville enfatiza que é uma organização de componentes com responsabilidades definidas



O que é Arquitetura?

Arquitetura = “como o sistema é organizado por dentro”



Por que Arquitetura é importante?

Segundo Pressman:

- Facilita comunicação entre equipe
- Define decisões críticas desde o início
- Reduz riscos do projeto
- Serve como modelo do sistema



Por que Arquitetura é importante?

Segundo Sommerville Impacta diretamente:

- Desempenho
- Segurança
- Disponibilidade
- Manutenção



Quando ela aparece?

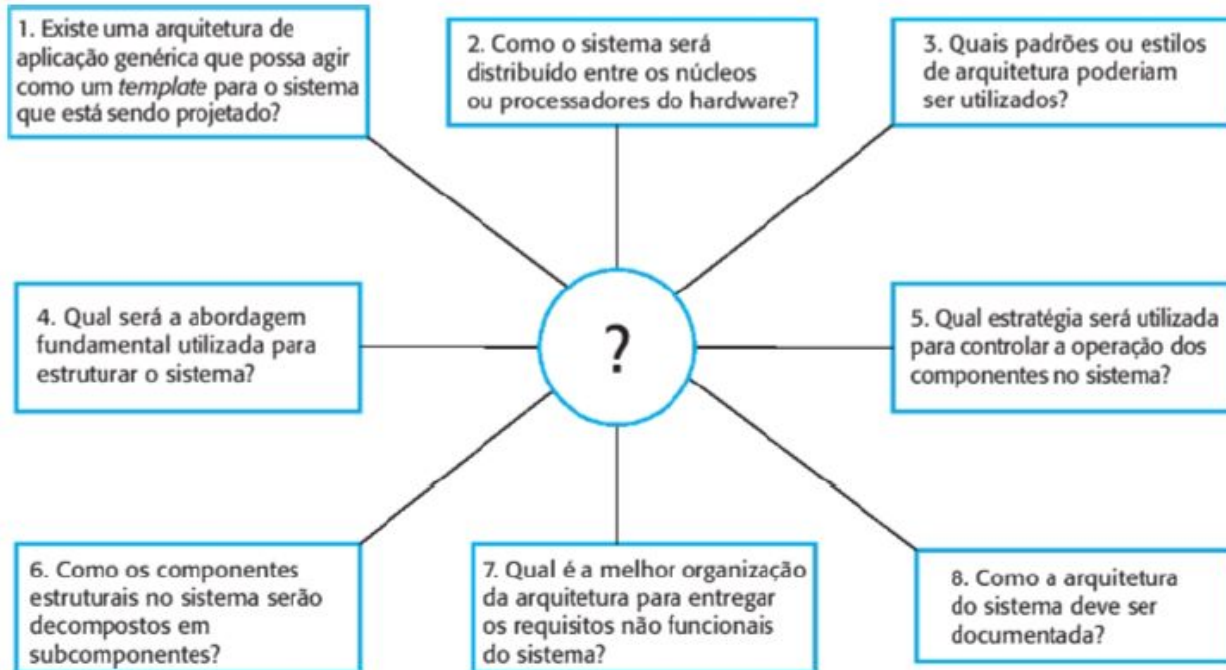
A arquitetura surge logo após os requisitos:

→ Requisitos → Arquitetura → Projeto detalhado → Implementação

Importante:

- A arquitetura não é código
- É uma abstração de alto nível

Decisões Arquiteturais





Visões de Arquitetura

Tipos de Visões:

- Visão Lógica → funcionalidades
- Visão de Processo → execução e concorrência
- Visão de Desenvolvimento → organização do código
- Visão Física → infraestrutura/hardware



Visões de Arquitetura

Tipos de Visões:

- Visão Lógica → mostra as abstrações fundamentais do sistema como objetos ou classes. Nesse tipo de visão, deve ser possível relacionar os requisitos do sistema às suas entidades.
- Visão de Processo → execução e concorrência
- Visão de Desenvolvimento → organização do código
- Visão Física → infraestrutura/hardware



Visões de Arquitetura

Tipos de Visões:

- Visão Lógica → funcionalidades
- Visão de Processo → mostra como, no tempo de execução, o sistema é composto de processos que interagem. Essa visão é útil para fazer julgamentos sobre características não funcionais do sistema, como o desempenho e a disponibilidade.
- Visão de Desenvolvimento → organização do código
- Visão Física → infraestrutura/hardware



Visões de Arquitetura

Tipos de Visões:

- Visão Lógica → funcionalidades
- Visão de Processo → execução e concorrência
- Visão de Desenvolvimento → mostra como o software é decomposto para desenvolvimento; isto é. mostra a divisão do software em componentes que são implementados por um único desenvolvedor ou time de desenvolvimento. Essa visão é útil para gerentes e programadores de software.
- Visão Física → infraestrutura/hardware



Visões de Arquitetura

Tipos de Visões:

- Visão Lógica → funcionalidades
- Visão de Processo → execução e concorrência
- Visão de Desenvolvimento → organização do código
- Visão Física → mostra o hardware do sistema e como os componentes de software estão distribuídos pelos processadores no sistema. Essa visão é útil para os engenheiros de sistema que estão planejando uma implantação do sistema.

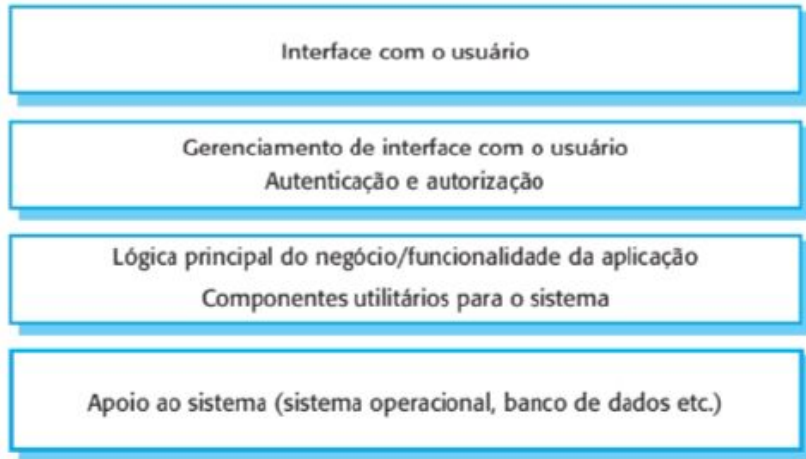


Estilos Arquiteturais / Padrões de Arquitetura

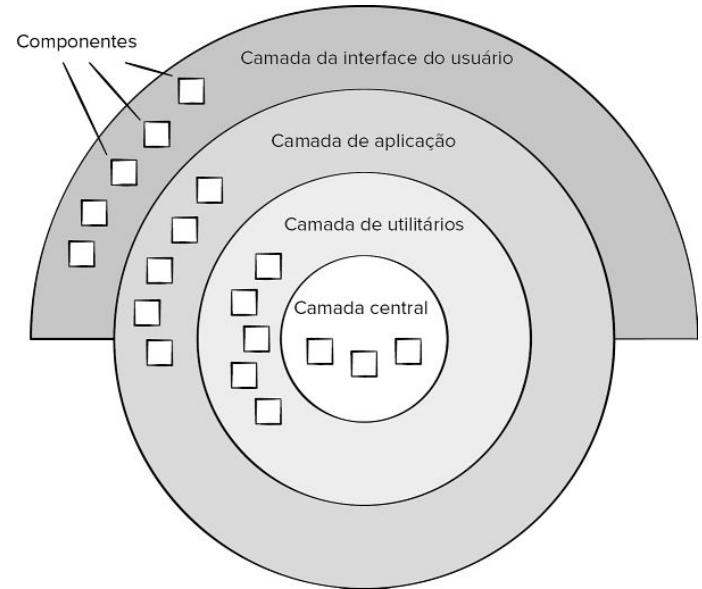
O que são?

Um padrão geral de organização do sistema

Arquitetura em camadas



Fonte: (Sommerville, 2018)



Fonte: (Pressman, 2021)



Arquitetura em camadas

Organiza o sistema em camadas, com funcionalidade associada a cada uma. Uma camada fornece serviços para a camada acima dela, então as camadas nos níveis mais inferiores representam os serviços essenciais que tendem a ser utilizados em todo o sistema



Arquitetura em camadas

Quando é utilizado:

Utilizado quando se cria novos recursos em sistemas existentes; quando o desenvolvimento é distribuído por vários times, cada um deles responsável por uma camada de funcionalidade; quando há necessidade de segurança da informação Security) em múltiplos níveis.



Arquitetura em camadas

Vantagens:

Permite a substituição de camadas inteiras contanto que a interface seja mantida. Recursos redundantes, como a autenticação, podem ser fornecidos em cada camada para aumentar a dependabilidade do sistema.

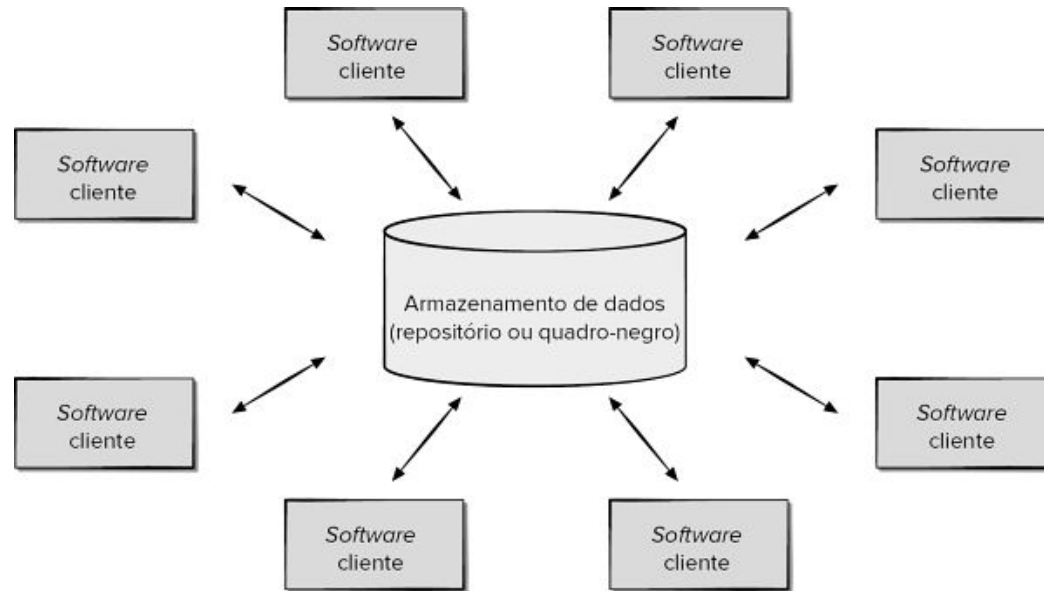


Arquitetura em camadas

Desvantagens:

Na prática, muitas vezes é difícil proporcionar uma separação clara entre as camadas, de modo que camadas dos níveis mais altos podem ter de interagir diretamente com as dos níveis mais baixos em vez das imediatamente inferiores a elas. O desempenho pode ser um problema por causa dos múltiplos níveis de interpretação de uma requisição de serviço à medida que essa requisição é processada em cada camada.

Arquitetura de repositório/centralizada em dados



Fonte: (Pressman, 2021)



Arquitetura de repositório/centralizada em dados

Todos os dados em um sistema são gerenciados em um repositório central que é acessível a todos os componentes do sistema. Os componentes não interagem diretamente, apenas por meio do repositório.



Arquitetura de repositório/centralizada em dados

Quando é utilizado:

Esse padrão deve ser usado em um sistema no qual são gerados grandes volumes de informação que precisam ser armazenados por muito tempo. Também é possível usá-lo nos sistemas dirigidos por dados, cuja inclusão no repositório dispara uma ação ou uma ferramenta.



Arquitetura de repositório/centralizada em dados

Vantagens:

Os componentes podem ser independentes; eles não precisam saber da existência dos outros componentes. As mudanças feitas por um componente podem ser propagadas para todos os demais. Todos os dados podem ser gerenciados de modo consistente (por exemplo, backups feitos ao mesmo tempo), já que estão todos em um só lugar.



Arquitetura de repositório/centralizada em dados

Desvantagens:

O repositório é um único ponto de falha, então os problemas no repositório afetam o sistema inteiro. Pode haver ineficiências em organizar toda a comunicação por meio do repositório. Pode ser difícil distribuir o repositório por vários computadores..

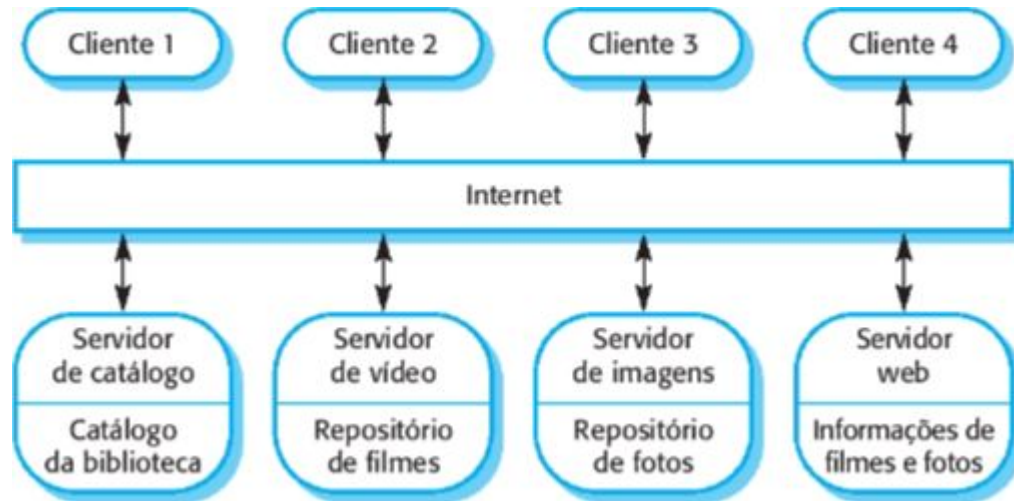


Arquitetura cliente-servidor

Um sistema que segue o padrão cliente-servidor é organizado como um conjunto de serviços e servidores associados e de clientes que acessam e usam esses serviços. Os principais componentes desse modelo são:

1. Um conjunto de servidores que oferecem serviços para outros componentes.
2. Um conjunto de clientes que demanda os serviços oferecidos pelos servidores.
3. Uma rede que permite que os clientes acessem esses serviços.

Arquitetura cliente-servidor



Fonte: (Sommerville, 2018)



Arquitetura cliente-servidor

Em uma arquitetura cliente-servidor, o sistema é apresentado como um conjunto de serviços, e cada serviço é fornecido por um servidor separado. Os clientes são usuários desses serviços e acessam os servidores para usá-los.



Arquitetura cliente-servidor

Quando é utilizado:

Utilizado quando os dados em um banco de dados compartilhado têm de ser acessados a partir de diversos locais. Como os servidores podem ser replicados, eles também podem ser utilizados quando a carga em um sistema for variável.



Arquitetura cliente-servidor

Vantagens:

A principal vantagem desse modelo é que os servidores podem ser distribuídos em rede. A funcionalidade geral (por exemplo, um serviço de impressão) pode estar disponível para todos os clientes e não precisa ser implementada por todos os serviços.



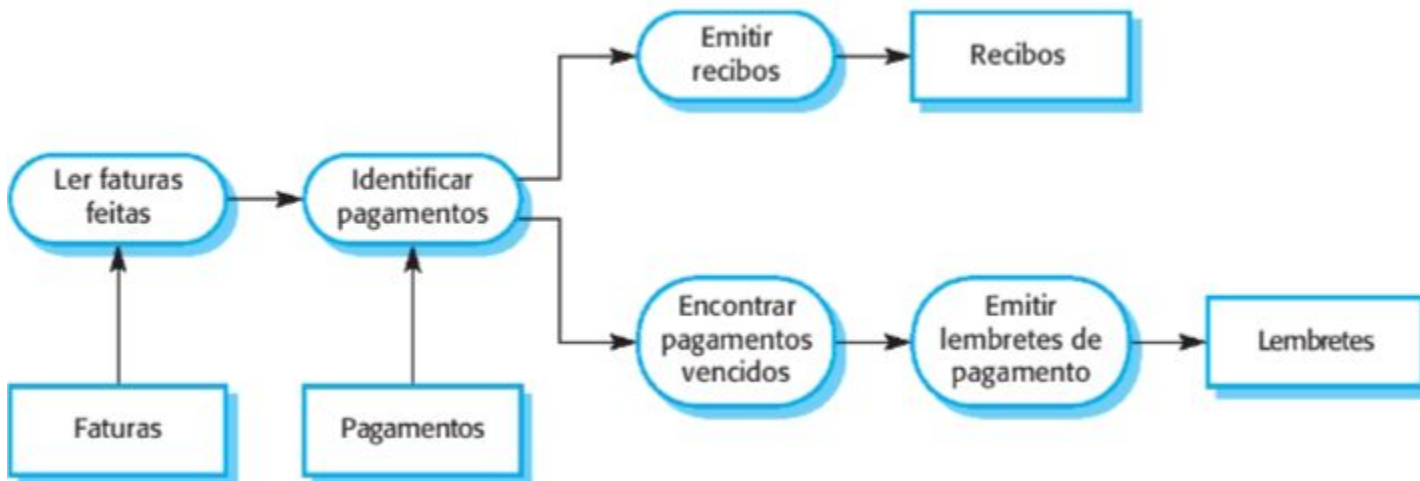
Arquitetura cliente-servidor

Desvantagens:

Cada serviço é um único ponto de falha e, portanto, é suscetível a ataques de negação de serviço ou a falhas no servidor. O desempenho pode ser imprevisível porque depende da rede e também do sistema. Podem surgir problemas de gerenciamento se os servidores forem de propriedade de organizações diferentes.

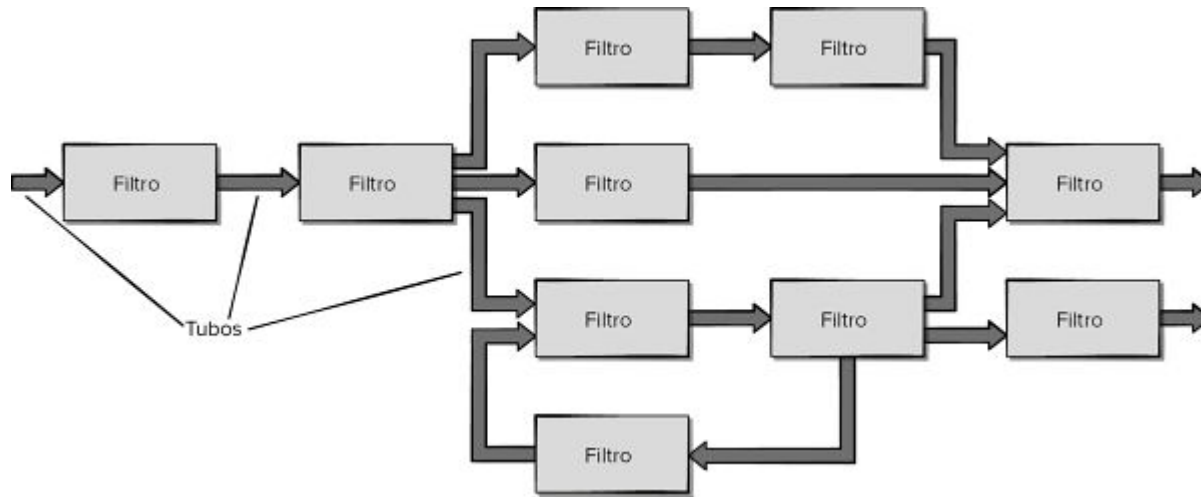
Arquitetura duto e filtro

FIGURA 6.15 Exemplo da arquitetura duto e filtro.



Fonte: (Sommerville, 2018)

Arquitetura duto e filtro



Fonte: (Pressman, 2021)



Arquitetura duto e filtro

O processamento dos dados em um sistema é organizado de modo que cada componente de processamento (filtro) é discreto e executa um tipo de transformação dos dados. Os dados fluem (como em um duto) de um componente para outro para serem processados.



Arquitetura duto e filtro

Quando é utilizado:

Utilizado frequentemente nas aplicações de processamento de dados (tanto baseadas em lotes quanto em transações), em que as entradas são processadas em estágios separados, gerando saídas relacionadas.



Arquitetura duto e filtro

Vantagens:

Fácil de entender e permite o reúso de transformações. O estilo do fluxo de trabalho corresponde à estrutura de muitos processos de negócio. A evolução por meio da adição de transformações é direta. Pode ser implementado como um sistema sequencial ou concorrente.

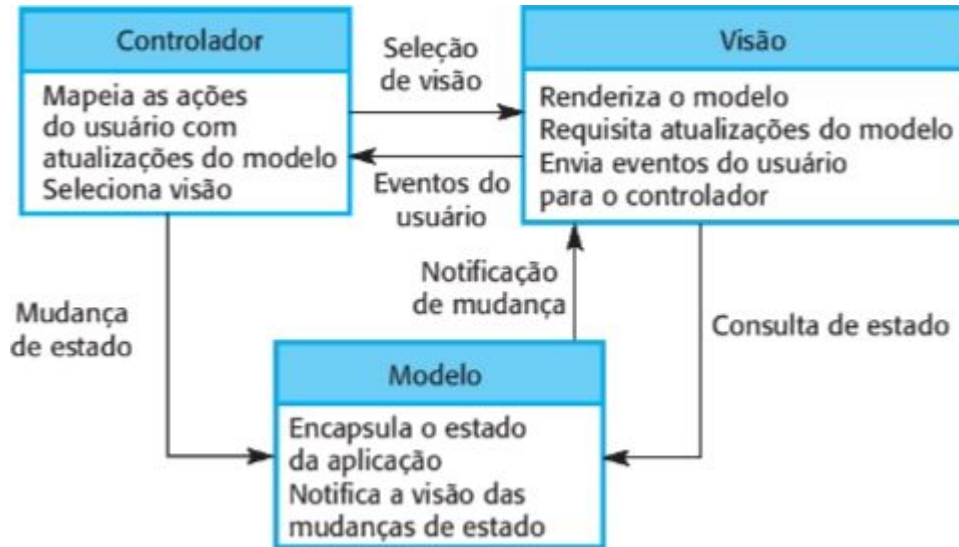


Arquitetura duto e filtro

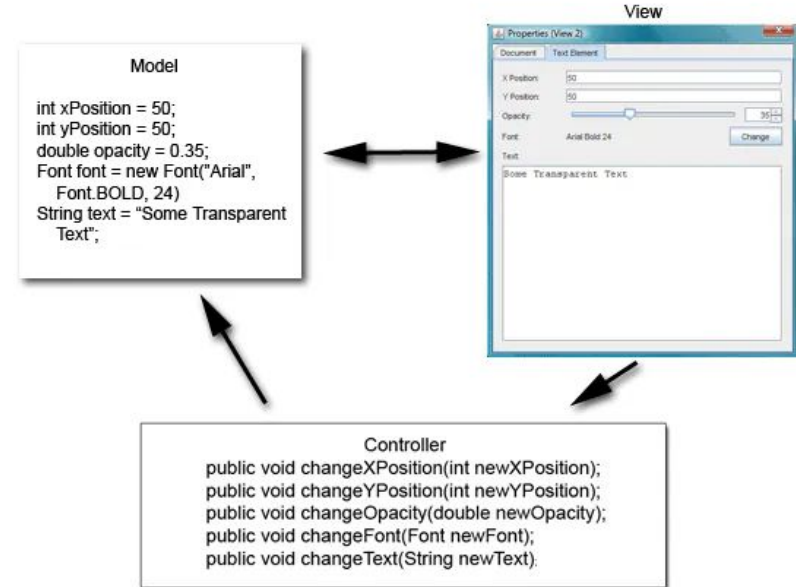
Desvantagens:

O formato da transferência de dados tem de ser acordado entre as transformações que se comunicam. Cada transformação deve analisar sua entrada e devolver a saída para a forma acordada. Isso aumenta a sobrecarga do sistema e pode significar que é impossível reusar componentes arquiteturais que usam estruturas de dados incompatíveis.

MVC - Modelo - Visão - Controlador

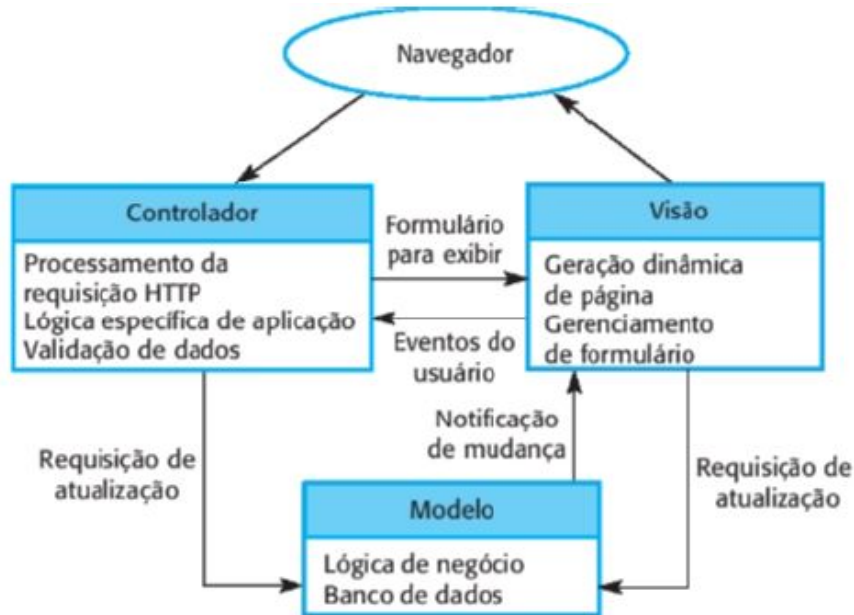


Fonte: (Sommerville, 2018)

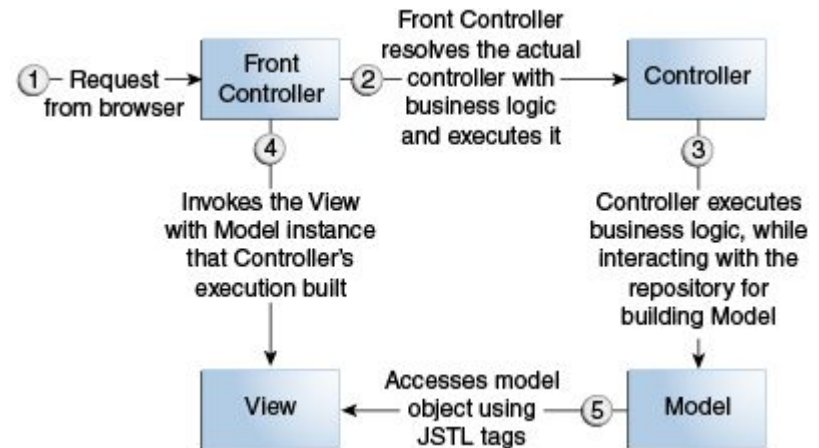


Fonte: (Java SE Oracle, 2026)

MVC Web - Modelo - Visão - Controlador

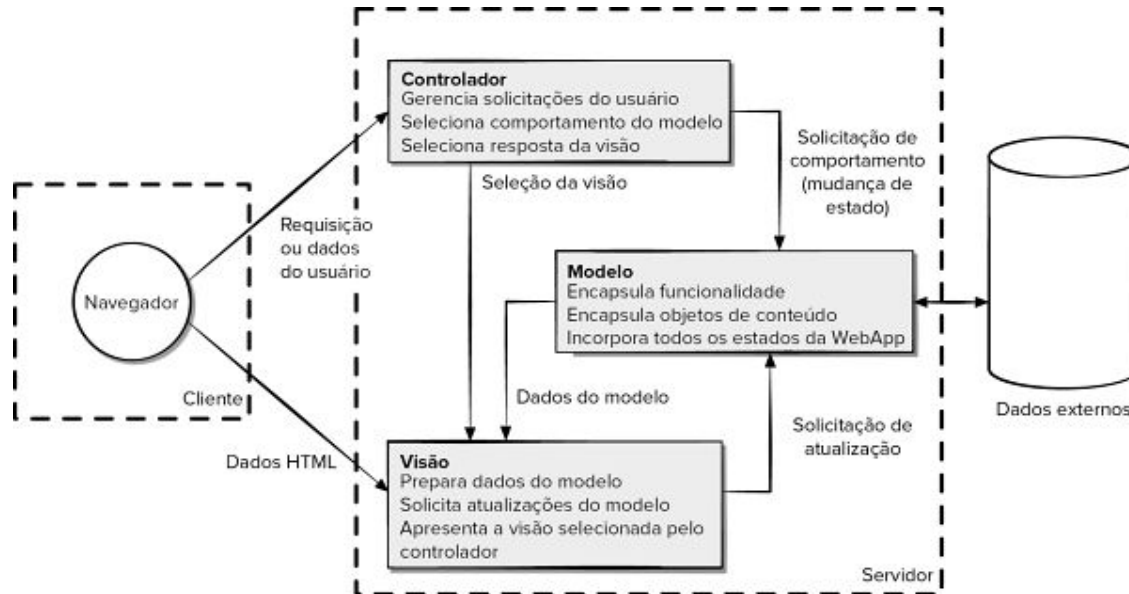


Fonte: (Sommerville, 2018)



Fonte: (Web Oracle, 2026)

MVC Web - Modelo - Visão - Controlador



Fonte: (Pressman, 2021)



MVC - Modelo - Visão - Controlador

Separa a apresentação e a interação dos dados do sistema. O sistema é estruturado em três componentes lógicos que interagem entre si.

- O componente **Modelo** gerencia os dados do sistema e as operações a eles associadas.
- O componente **Visão** define e gerencia como os dados são apresentados ao usuário.
- O componente **Controlador** gerencia a interação do usuário (por exemplo, pressionamento de teclas, cliques de mouse etc.) e passa essas interações para Visão e Modelo.



MVC - Modelo - Visão - Controlador

Quando é utilizado:

É utilizado quando há várias maneiras de visualizar e interagir com os dados. Também é utilizado quando os requisitos futuros para interação e apresentação dos dados são desconhecidos.



MVC - Modelo - Visão - Controlador

Vantagens:

Permite que os dados sejam alterados independentemente de sua representação e vice-versa. Apoia a apresentação dos mesmos dados de maneiras diferentes, exibindo as alterações feitas em uma representação em todas as demais.



MVC - Modelo - Visão - Controlador

Desvantagens:

Pode envolver mais código e aumentar sua complexidade quando o modelo de dados e as interações forem simples.



Arquiteturas de Aplicações

O que são Arquiteturas de Aplicações?

Arquiteturas de aplicações são modelos genéricos reutilizáveis que descrevem a estrutura de sistemas dentro de um domínio específico.

Ou seja:

São “modelos prontos” que podem ser adaptados para desenvolver sistemas semelhantes.



Sistemas de processamento de transações

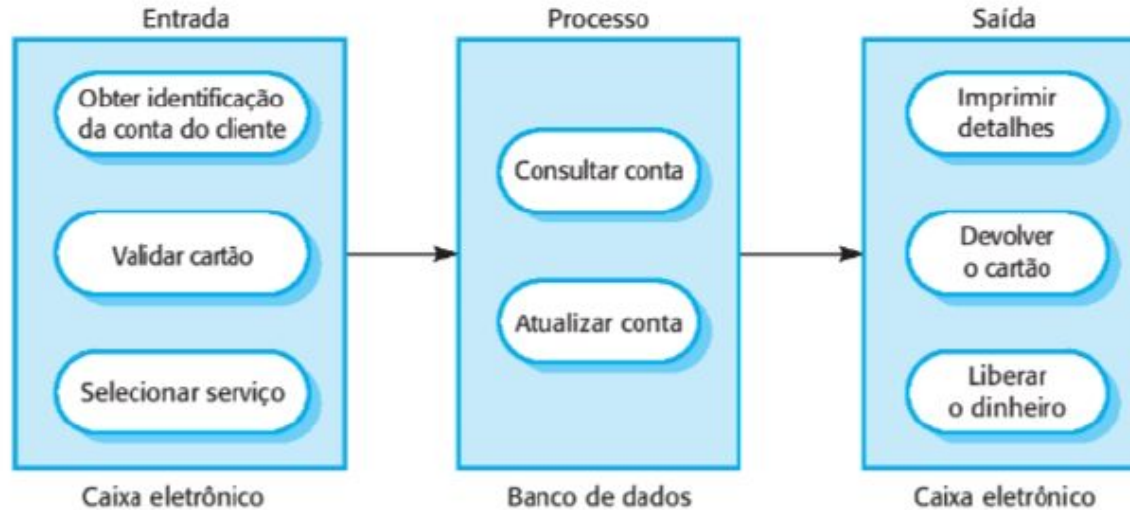
Os sistemas de processamento de transações são concebidos para processar requisições de um usuário por informação em um banco de dados ou requisições para atualizar um banco de dados



Sistemas de processamento de transações

Um exemplo de transação de banco de dados é a solicitação de um cliente para retirar dinheiro de uma conta bancária usando um caixa eletrônico. Isso envolve conferir se há saldo na conta do cliente, modificar o saldo relativo à quantidade retirada e enviar comandos para o caixa eletrônico entregar o dinheiro. Enquanto todas essas etapas não forem concluídas, a transação está incompleta e o banco de dados das contas de cliente não é alterado.

Sistemas de processamento de transações



Fonte: (Sommerville, 2018)



Sistemas de informação

Um sistema de informação permite o acesso controlado a uma grande base de informações, como um catálogo de uma biblioteca, uma tabela de horários de voos ou os registros dos pacientes em um hospital. Os sistemas de informação quase sempre são sistemas web. cuja interface com o usuário é acessada por meio de um navegador.



Sistemas de informação



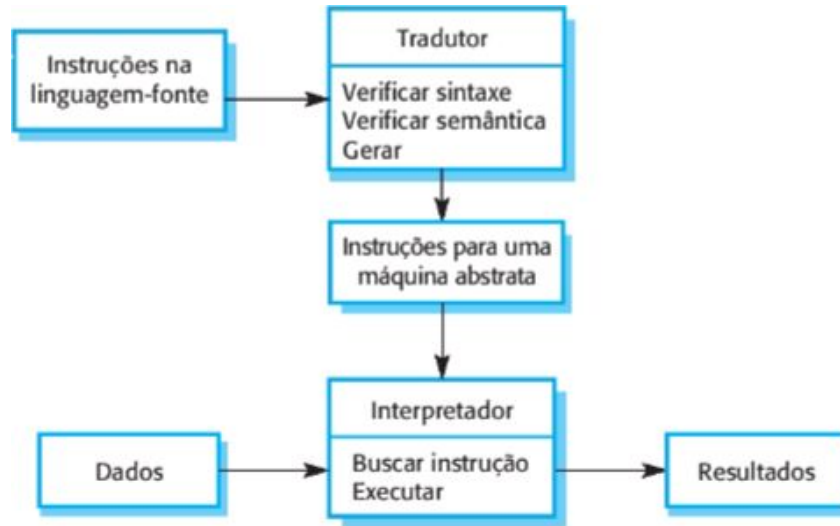
Fonte: (Sommerville, 2018)



Sistemas de processamento de linguagem

Os sistemas de processamento de linguagem traduzem uma linguagem para uma representação alternativa e, no caso das linguagens de programação, também podem executar o código resultante. Os compiladores traduzem uma linguagem de programação em código de máquina.

Sistemas de processamento de linguagem



Fonte: (Sommerville, 2018)



Referências

SOMMERVILLE, Ian. Engenharia de software. 10. ed. São Paulo: Pearson, 2019.

PRESSMAN, Roger S.; MAXIM, Bruce R. Engenharia de software: uma abordagem profissional. 8. ed. Porto Alegre: AMGH, 2016.

ORACLE. Developing WebCenter Sites Applications. Disponível em: <https://docs.oracle.com/middleware/12212/wcs/develop/GUID-C8899CBC-2EC1-4A25-A887-F8B9A868084D.htm#WBCSD8197> Acesso em: 02 fev. 2026.

ORACLE. Model-View-Controller (MVC) Design Pattern. Disponível em: <https://www.oracle.com/technical-resources/articles/javase/mvc.html> Acesso em: 02 fev. 2026.